VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3071 - Distributed Systems

ASSIGNMENT REPORT

*dunno what should be for the title

Advisor(s):	Thoại Nam	
Student(s):	Bùi Trần Quyền	ID 2353033
	Nguyễn Đức Phi Long	ID 2452689
	Trần Đức Phú	ID 2452971

HO CHI MINH CITY, NOVEMBER 2026

Contents

1 Introduction **5**
1.1 Background and motivation . . . 5
1.2 Problem statement . . . 5
1.3 Objectives . . . 5
1.4 Scope and limitations . . . 5
1.5 Report structure . . . 5

2 Background and Theory **6**
2.1 Definition . . . 6
2.1.1 Basic Crawling Process . . . 6
2.1.2 Main Components . . . 6
2.1.3 Crawling Policies . . . 7
2.1.4 Why Distribute the Crawler? . . . 7
2.2 Concepts and Knowledge from the course . . . 7
2.2.1 Architecture and Communication . . . 7
2.2.2 Task Distribution and Concurrency . . . 8
2.2.3 Fault Tolerance and Consistency . . . 8
2.2.4 Scalability Metrics . . . 9
2.3 Related work or existing systems . . . 10

3 System Design **11**
3.1 Requirements . . . 11
3.2 Architecture and components . . . 12
3.3 Communication and key mechanisms . . . 15
3.4 Design decisions and trade-offs . . . 16

4 Implementation **18**
4.1 Implementation of the main components . . . 18
4.2 Deployment and challenges . . . 18

5 Experiments and Evaluation **19**
5.1 Experimental setup . . . 19
5.2 Scalability and performance . . . 19
5.3 Fault tolerance and duplicate handling . . . 19
5.4 Discussion . . . 19

6 Conclusion **20**
6.1 Summary against the objectives 20
6.2 Limitations and future work 20

7 References **21**



1 Introduction

1.1 Background and motivation

1.2 Problem statement

1.3 Objectives

1.4 Scope and limitations

1.5 Report structure

2 Background and Theory

2.1 Definition

A web crawler (also called a spider or bot) is a program that automatically browses the World Wide Web. Starting from a set of **seed URLs**, it downloads web pages, extracts data and hyperlinks from them, and follows those links to discover further pages. Search engines use crawlers to build their indexes, and other common applications include price monitoring, data collection for machine learning, and website archiving.

2.1.1 Basic Crawling Process

A crawler repeats the following loop until its frontier is empty or a stopping condition is reached:

1. Take a URL from the **frontier**, the queue of URLs waiting to be crawled.
2. Download the page over HTTP.
3. Parse the HTML and extract useful content (for example the title and text) and the outgoing links.
4. Normalize the extracted links and discard those that have already been seen.
5. Add the new links to the frontier, store the results, and repeat.

2.1.2 Main Components

- **Frontier:** the queue of URLs to be crawled.
- **Fetcher:** the HTTP client that downloads pages.
- **Parser:** extracts content and links from the HTML.
- **Duplicate detector:** a registry of URLs that were already discovered or processed.
- **Storage:** persists the crawled data and the crawl status.



2.1.3 Crawling Policies

A practical crawler must follow several policies:

- **Politeness:** respect the `robots.txt` file of each site and limit the request rate per domain so that the target server is not overloaded.
- **Depth and URL limits:** bound the crawl to avoid infinite crawling caused by endlessly generated links.
- **URL normalization:** treat different spellings of the same address as a single URL.
- **Robustness:** handle timeouts, redirects, malformed HTML, and unavailable servers.

2.1.4 Why Distribute the Crawler?

A single machine is limited by its network bandwidth, CPU, and memory. In addition, websites respond at very different speeds, so one slow download can delay all following work in a sequential crawler. Distributing the crawl over several workers increases throughput, but it introduces typical distributed-systems problems:

- How are URLs divided among the workers?
- How can the system avoid crawling the same page more than once?
- What happens when a worker, or the coordinator, fails?
- At which point does adding more workers stop improving performance?

These questions are the focus of this project.

2.2 Concepts and Knowledge from the course

This section presents the distributed-systems concepts from the course that are needed to design and evaluate the distributed crawler. Table 2.1 at the end of the section relates them to the problems of the crawler.

2.2.1 Architecture and Communication

A distributed system consists of independent nodes that communicate only by passing messages over a network, with no shared memory and no shared clock. The network may delay or drop messages, and nodes may crash. In this project, the nodes are a **scheduler**,

several **crawler workers**, a **message broker** (Apache Kafka), and a **database** (PostgreSQL). The failure model considered is the crash of a worker; the failure of a broker or of the scheduler is treated as an extension.

Instead of a direct request–response style (RPC or REST) between a coordinator and its workers, the system uses the **producer–consumer** pattern over a message queue, as covered in the data-streaming part of the course. The scheduler *produces* crawl tasks into a Kafka topic and the workers *consume* them; the workers in turn produce newly discovered URLs into another topic that the scheduler consumes. The broker decouples the components: they do not need to know each other's address, and workers can be added or removed without changing the scheduler. Kafka stores messages in **topics**, each split into **partitions**, and consumers that share a **consumer group** divide the partitions among themselves. The workers run as containers (pods) managed by Kubernetes.

2.2.2 Task Distribution and Concurrency

With **static allocation**, each worker receives a fixed set of URLs in advance, so fast workers may sit idle while slow ones are still busy. With **dynamic allocation**, a worker takes a new task whenever it becomes free, which balances the load automatically. A Kafka consumer group behaves like dynamic allocation at the granularity of partitions: each partition is read by exactly one consumer of the group, so the number of useful workers is bounded by the number of partitions. Messages are keyed by domain, so that all URLs of one domain are handled by the same worker, which keeps per-domain politeness (rate limiting) simple.

If two workers discover the same URL at almost the same time, both report it to the scheduler. Checking whether a URL is known and registering it must therefore be a single **atomic** operation; otherwise a **race condition** may create the same task twice. In this project the scheduler is the only component that decides whether a URL is new, and it relies on the atomicity of a database operation (a unique constraint on the URL in PostgreSQL) instead of application-level locks.

2.2.3 Fault Tolerance and Consistency

Failures of workers are detected through the consumer-group heartbeat and session timeout of Kafka: when a worker stops responding, its partitions are reassigned to the remaining workers. Kubernetes additionally restarts a failed pod. Because a worker commits its position in the topic only after it has processed a task, a task that was in progress when the worker crashed is delivered again. The system therefore provides **at-least-once** deliv-



ery. Since a task may be processed twice, storing results must be **idempotent**: processing the same result twice must not create duplicate records. Failed tasks can additionally be retried and, after too many failures, moved to a dead-letter queue.

The CAP theorem states that during a network partition a distributed system must choose between consistency and availability. The central URL registry in PostgreSQL favors **consistency**: if the scheduler cannot reach the database, it stops issuing new tasks instead of risking duplicates, so availability is reduced during the failure. This trade-off is discussed in the design chapter.

2.2.4 Scalability Metrics

The scalability and the reliability of the system are evaluated with the following metrics:

- total crawling time and throughput (pages per second);
- speedup and efficiency compared with the single-machine baseline (1, 2 and 3 workers);
- recovery time after a worker failure and the number of lost or duplicated tasks.

With T_1 the running time on one worker and T_p the running time on p workers, the speedup and the efficiency are

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p}.$$

Amdahl's law bounds the achievable speedup when a fraction f of the work is sequential (here, mainly the scheduler and the database):

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}.$$

Summary of concepts. Table 2.1 relates the problems of the crawler to the concepts of the course.

Problem in the crawler	**Concept**	**Course topic**
Scheduler and workers exchange messages	Producer–consumer, message queue (Kafka)	Communication
Many workers share the crawl	Partitions, consumer groups, dynamic allocation, load balancing	Distributed computing
Two workers discover the same URL	Race condition, atomic operation (unique constraint)	Transactions, coordination
A task is delivered twice	At-least-once delivery, idempotency	Transactions
A worker crashes during a task	Heartbeat, rebalancing, pod restart, retry	Replication, fault tolerance
A broker or the scheduler fails	Replicated partitions, persistent state	Replication, consensus
Shared visited-URL set	Consistency, CAP theorem	Introduction

Table 2.1: Crawler problems and the related distributed-systems concepts.

2.3 Related work or existing systems



3 System Design

3.1 Requirements

The design follows the principle "build once, deploy anywhere, scale by configuration": the same container image must run on any machine, and the capacity of the system must be changed by configuration (the number of pods or nodes) and not by changing the code.

Functional requirements.

FR1 Accept a set of seed URLs and crawl from them within configurable limits (maximum depth and maximum number of pages per domain).

FR2 Distribute the URLs to several crawler workers as tasks through Kafka.

FR3 For each task, download the page, extract its content and outgoing links, and respect `robots.txt`.

FR4 Store the crawled data and the status of each URL in PostgreSQL.

FR5 Send the newly discovered URLs back to the scheduler, which must avoid creating unnecessary duplicate tasks.

FR6 Process again the tasks that were not completed when a worker failed.

Criterion	Requirement
Correctness	Crawled data is collected and stored correctly; duplicate URLs are kept to a minimum.
Distribution	Several crawler workers process the tasks distributed by Kafka.
Scalability	The number of pods and worker nodes can be increased without modifying the application code.
Fault tolerance	A failed worker is replaced, and the tasks it did not finish are processed again.
Performance	Performance figures of the distributed crawler can be compared with the single-machine baseline under the same conditions.
Reproducibility	Another person can set up and run the system with the provided instructions and scripts, using the same library versions and container images.
Documentation	Architecture diagram, run instructions, report, and demo video are provided.

Table 3.1: Non-functional requirements.

Non-functional requirements.

3.2 Architecture and components

The system is built from four kinds of nodes, which communicate only through messages and the database: a **scheduler**, several **crawler workers**, a **Kafka** cluster, and a **PostgreSQL** database. Figure 3.1 shows the overall architecture.

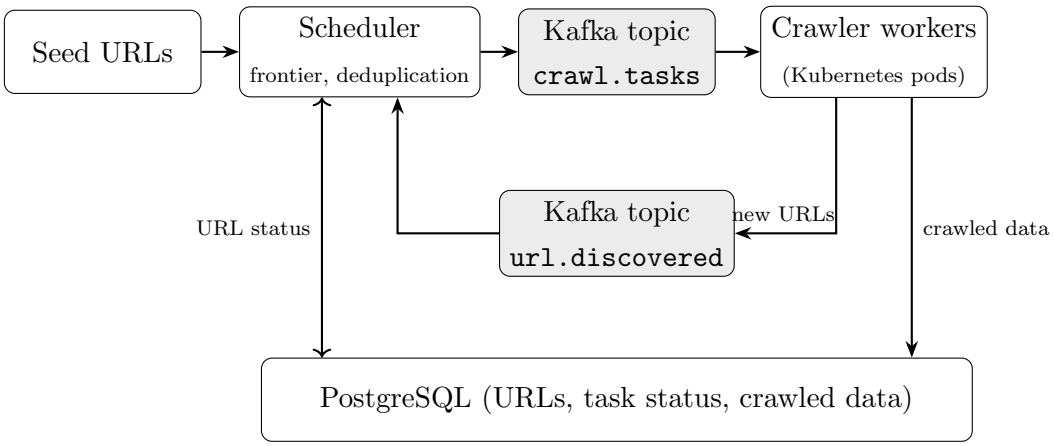


Figure 3.1: Overall architecture of the distributed crawler.

The data flow of the system is as follows:

1. The seed URLs are loaded into the scheduler.
2. The scheduler normalizes each URL, checks in PostgreSQL whether it is already known, and only publishes the new ones as tasks to the topic `crawl.tasks`.
3. The crawler workers, which run as several Kubernetes pods, consume the tasks from Kafka.
4. Each worker downloads the page, parses it, and extracts the content and the outgoing links.
5. The crawled data is stored in PostgreSQL, and the new URLs are published to the topic `url.discovered`.
6. The scheduler consumes `url.discovered` and repeats from step 2, until the depth and page limits are reached.

Components. Table 3.2 lists the responsibilities of the components and the corresponding modules of the code.

Component	Responsibility	Module
Scheduler	Normalizes URLs, registers new URLs atomically, applies the depth and page limits, and publishes tasks to Kafka	`scheduler/main.py`, `frontier.py`
Crawler worker	Consumes tasks, checks `robots.txt` and the request rate, downloads and parses the page, stores the result, and publishes new URLs	`crawler/main.py`, `fetcher.py`, `parser.py`, `politeness.py`
Storage	Database schema and all PostgreSQL operations	`storage/schema.sql`, `repository.py`
Kafka	Two brokers; the topics `crawl.tasks` and `url.discovered`	`k8s/kafka.yaml`
Common code	Configuration, Kafka producer/consumer, URL normalization	`common/`
Seed loader	Inserts the initial URLs	`scripts/seed.py`
Baseline crawler	Single-process crawler that reuses the same fetcher and parser; used for comparison	`crawler/`

Table 3.2: Components of the system and their modules.

Deployment view. All services run on a Kubernetes cluster of two virtual machines. The scheduler and the crawler workers are Kubernetes Deployments (two to three crawler pods), PostgreSQL runs with a persistent volume, and Kafka runs with two brokers. Adding workers means increasing the number of replicas, or adding a node to the cluster, without changing the Python code.

Design patterns. The design uses the **Producer–Consumer** pattern (scheduler and workers through Kafka), the **Repository** pattern (all database access goes through one module, so the other modules do not contain SQL), and the **Strategy** pattern for re-

placeable behaviours.

3.3 Communication and key mechanisms

Topics and messages. Table 3.3 describes the two Kafka topics.

Topic	Direction	Content	Message key
`crawl.tasks`	Scheduler → workers	URL to crawl, its depth, and its domain	Domain
`url.discovered`	Workers → scheduler	Links found on a page, with the depth of that page	–

Table 3.3: Kafka topics used by the system.

```
{"url": "https://example.com/a", "depth": 2, "domain": "example.com"}
```

URL normalization and deduplication. Different spellings of the same address (for example upper-case host names, fragments, or trailing slashes) are first converted to one canonical form. The scheduler is the only component that decides whether a URL is new: it registers the canonical URL in PostgreSQL, where the URL has a unique constraint. Registering a URL that already exists is ignored by the database, and only the URLs that were really inserted are published as tasks. Because the check and the insertion form one atomic database operation, two workers that find the same link at the same time cannot create two tasks.

Task status. Each URL has a status in PostgreSQL, for example `pending` (registered and sent to Kafka), `done` (crawled and stored), and `failed` (all attempts failed). The status makes it possible to find unfinished work and to count the results of the experiments.

Delivery guarantee and recovery. A worker commits its position in the topic (the Kafka offset) only after the result is stored in PostgreSQL. If the worker crashes before that, the task is delivered again to another worker after Kafka reassigns the partitions, and Kubernetes starts a new pod. This gives **at-least-once** delivery. Because the result is stored by URL (an insert or update of the same record), processing a task twice does

not create duplicate data. A failed task is retried a limited number of times, and every failure is logged.

Politeness and limits. A worker reads `robots.txt` of each domain and waits a minimum time between two requests to the same domain. Since the message key of `crawl.tasks` is the domain, all URLs of one domain go to the same partition and therefore to the same worker, so each worker can enforce this delay locally without coordination with the others. The scheduler stops publishing URLs whose depth exceeds the maximum depth or whose domain has reached its page limit.

3.4 Design decisions and trade-offs

Table 3.4 summarizes the main design decisions.

Decision	Alternative	Reason and trade-off
Kafka as task queue	REST coordinator–worker	Kafka decouples the components, buffers tasks, and reassigns partitions automatically when a worker fails; the price is the effort of operating Kafka and an extra network hop.
Central deduplication in scheduler and PostgreSQL	Deduplication inside every worker	One place decides, so consistency is simple; but the scheduler and the database can become a bottleneck when workers increase (Amdahl's law). In terms of the CAP theorem the system prefers consistency: if the database is unreachable, no new tasks are created.
Partition by domain	Random distribution	Politeness is easy to enforce locally; but the parallelism is limited by the number of partitions and domains, and one very large domain keeps a single worker busy.
At-least-once delivery with idempotent storage	Exactly-once processing	Much simpler to implement; a task may be processed twice, but this does not change the stored data.
Kubernetes	Processes started manually	Scaling by replicas and automatic restart of failed pods; but setting up the cluster (kubeadm on several VMs) takes effort.

Table 3.4: Main design decisions and their trade-offs.

Out of scope. The first version does not include a Dead Letter Queue or a Transactional Outbox, distributed rate limiting across workers, a monitoring dashboard, auto-scaling with KEDA, or a production-grade fault-tolerant Kafka setup. They are mentioned as future work in the conclusion.

4 Implementation

Technology	Role
Python 3.10, `uv`	Programming language and project/dependency management
`requests`, BeautifulSoup	Download HTML and extract content and links
Apache Kafka, `confluent-kafka`	Message broker that distributes crawl tasks; Python producer/consumer
PostgreSQL, `psycopg`	Stores URLs, task status, and crawled data
Docker, containerd	Build container images; container runtime in Kubernetes
Kubernetes (kubeadm) on Ubuntu Server 24.04	Runs and scales the scheduler and the worker pods on several VMs
Ansible, Helm	Automated VM setup; packaged application deployment
Docker Compose	Local development environment
GitHub, GitHub Actions	Version control, pull requests, CI
`pytest`, `ruff`	Unit/integration tests; linting and formatting

Table 4.1: Technology stack of the project.

4.1 Implementation of the main components

4.2 Deployment and challenges

5 Experiments and Evaluation

5.1 Experimental setup

5.2 Scalability and performance

5.3 Fault tolerance and duplicate handling

5.4 Discussion

6 Conclusion

6.1 Summary against the objectives

6.2 Limitations and future work



7 References